모바일 AI 및 SW 기술 조사 보고서

가격 비교 기반 장보기 투두 앱 얼마였지

과목	고급모바일프로그래밍
팀 구성원	2191298 김종현 · 2271494 이재영 · 2311122 신지현
문서 정보	조사 v1.0 · 2026년 10월 9일
문서 범위	크로스플랫폼 · 모바일 AI OCR · 로컬 저장 · 모바일 백엔드

조사 목적

얼마였지는 장보기 목록과 영수증 기반 가격 기록을 연결하는 앱이다. 목록 관리와 기기 저장, 기기 간 동기화, 종이 메모 인식을 기본 범위로 두고 영수증 인식과 구매 가격 비교를 확장 범위로 둔다.

각 기능에 필요한 기술의 특성, 구현 방법, 추가 개발 범위를 비교한다. 이후 작성할 모바일 AI 및 SW 기술 조사 및 채택기술 보고서의 근거 자료로 활용하되, 이 단계에서는 채택 기술이나 서비스 순위를 확정하지 않는다.

조사 대상

분야	비교 대상	확인할 내용
모바일 SW	Flutter · React Native	화면 구성, 카메라와 플랫폼 기능 연결
모바일 AI	ML Kit · Tesseract	한국어 이미지 OCR, 모델 배포, 후처리
로컬 저장	설정 저장소 · SQLite	재실행 복원, 구매 확정의 원자적 저장
백엔드	Firebase · Supabase · AWS Amplify	사용자 구분, 동기화, 구매 기록, 가격 조회

자료와 판단의 구분

프로젝트 제안서, 요구사항 분석서 v1.0, UI 프로토타입 설계서 v1.0과 모바일 백엔드 서비스 강의 자료를 기준으로 삼았다. 기존 크로스플랫폼 조사 v36.0의 비교 범위를 연결하고, 제품 기능은 공식 개발 문서와 패키지 관리자의 문서에서 확인했다.

본문의 '구현 구성안'은 요구사항을 충족하기 위한 기술적 분석이다. 예제 코드는 설명용이며 앱 적용·서비스 배포·성능 시험 결과를 뜻하지 않는다. 성능 목표와 정책 제안은 요구사항의 미확정 상태를 유지한다.

1 조사 기준과 요구사항

기능을 제공한다는 설명과 프로젝트의 완료 조건을 구분해야 한다. 예를 들어 자동 동기화를 제공하는 서비스라도 충돌 시 사용자 선택, 삭제 항목의 부활 방지, 구매 확정의 중복 방지를 별도로 보장해야 한다.

조사 기능	관련 요구사항	구현 시 필요한 조건
장보기 목록과 저장	F-01~F-07 N-05	입력 검증, 저장 성공 후 화면 반영, 종료 후 미전송 변경 복원
사용자와 동기화	F-08~F-12 C-06	본인 데이터 접근, 계정별 대기 변경 분리, 양쪽 충돌 보존, 재전송 1회 처리
메모 OCR	F-13~F-16 N-07 · N-09	사진 입력, 기기 내 인식, 검토·수정·선택 후 추가, 임시본 정리
영수증 구매 확정	F-17~F-22	상품별 금액 검토, 목록 연결, 구매분·잔여분 분리, 가격과 목록을 함께 저장
가격 조회와 공유	F-23~F-28 C-03~C-05	동일 규격·할인 조건 비교, 구매일 기준 최근 기록, 공개 필드 제한·공유 해제

화면 흐름과 기술 경계

> 메모 촬영 → 기기 OCR → 메모 확인 → 로컬 목록 저장 → 서버 동기화

> 영수증 촬영 → 기기 OCR → 영수증 검토 → 구매 확정 → 가격 비교

UI 설계서는 핵심 화면 8종을 제시한다. 저장 실패 시 입력을 유지하고, 동기화 대기·진행·완료·실패를 구분한다. 원본 사진은 영구 저장·서버 전송하지 않으며, 사용자 사진 보관함의 원본을 삭제하지 않고 앱이 만든 임시 사본만 정리한다.

조사에서 유지할 범위

공동 목록 편집, 실시간 판매가·재고 보장, 결제·주문은 기본 범위 밖이다. 초기 확장 검증은 상품 5종·매장 2곳·영수증 양식 2종이라는 제안 규모를 기준으로 한다. 푸시 알림과 서버 사진 저장은 현 요구사항의 핵심 기능에 포함되지 않는다.

요구사항의 D-01~D-06은 운영 환경, 사용자 확인, 충돌, 부분 구매, 공개 정책, OCR 평가 기준을 팀에서 확정할 항목이다.

2 크로스플랫폼 모바일 SW 비교

Flutter는 Dart로 위젯을 구성하고 자체 렌더링 체계로 화면을 그린다. React Native는 React와 JavaScript 또는 TypeScript를 사용하며 기본 컴포넌트는 네이티브 화면 요소와 연결된다. 두 방식 모두 Android와 iOS 앱 개발에 활용할 수 있다. [1] [2]

비교 항목	Flutter	React Native
화면 구성	위젯·테마로 화면과 상태를 표현	React 컴포넌트와 상태로 화면 구성
사진 입력	image_picker로 촬영·앨범 선택	Expo 사용 시 expo-image-picker로 촬영·앨범 선택
기기 기능 연결	플러그인과 플랫폼 채널	네이티브 모듈·플랫폼별 구현
로컬 DB 연결	sqflite 등 SQLite 패키지	Expo 사용 시 expo-sqlite 등
개발 시 고려	Dart·위젯 구조와 플러그인별 플랫폼 설정 학습	React·네이티브 연동 학습, Expo 환경에 따른 모듈 사용 조건 확인

카메라와 DB 패키지 근거 [3] [4] [15] [16]

핵심 기능의 구현 방법

목록 추가와 완료: 입력한 상품명·수량을 검증하고, 로컬 저장 완료 후 화면 상태를 갱신한다. Flutter에서는 TextField와 버튼·체크박스, React Native에서는 TextInput과 버튼·목록 컴포넌트로 동작을 연결할 수 있다. 저장과 동기화의 책임은 화면 요소와 분리한다.

메모 사진 가져오기: 촬영·앨범 버튼에서 사진 입력 함수를 호출하고 취소는 정상 종료로 처리한다. 사진 경로를 OCR에 전달하며 권한 거부·인식 실패 시 직접 입력을 제공한다. Android의 앱 복구와 iOS 사용 목적 문구 등은 패키지별 설정을 확인한다. [3] [4]

비교 시 확인할 조건

같은 8개 화면, 같은 입력 자료와 기기에서 개발·검증해야 차이를 비교할 수 있다. 프레임워크의 지원 플랫폼 목록이 OCR 플러그인의 지원 범위를 넓혀 주지는 않는다. 한국어 OCR, 사진 처리, 접근성, 작은 화면과 키보드가 열린 상태를 각각 확인한다.

기존 크로스플랫폼 조사 v36.0은 목록 추가·완료와 사진 입력 예제를 다룬다. 이 문서는 해당 조사를 백엔드·AI와 연결하며, 프레임워크 간 성능 우열이나 개발 시간 차이를 측정값 없이 단정하지 않는다.

3 모바일 AI OCR 기술 비교

OCR은 사진에서 글자를 추출하는 단계다. 상품명·수량·매장·구매일·가격을 구분하는 작업과 동일 상품 판단은 별도의 후처리가 필요하다. OCR 결과를 바로 구매 기록으로 저장하면 영수증 합계나 할인 금액을 상품 가격으로 잘못 사용할 수 있다.

비교 항목	ML Kit Text Recognition v2	Tesseract
실행 방식	Android · iOS 기기에서 인식	앱에 엔진과 언어 모델을 포함해 로컬 실행 가능
한국어	한국어 문자 모델 지원	kor 언어 모델 제공
출력 활용	텍스트 블록·줄·요소와 위치 정보	텍스트 및 설정한 출력 형식의 위치 정보 활용
연동	네이티브 SDK 또는 Flutter 커뮤니티 플러그인	모바일용 래퍼·네이티브 빌드 연동 필요
조정 범위	SDK가 제공하는 문자 모델과 입력 옵션	언어 모델·분할 옵션·전처리 조정 가능
확인할 부담	한국어 의존성, 모델 준비, 플랫폼 버전	모델 배포, ABI별 빌드, 메모리·처리 시간

기술 특성의 공식 근거 [5] [6] [8] [9] [10]

손글씨 사진과 획 인식의 차이

ML Kit Digital Ink Recognition은 사용자가 화면에 그린 좌표·시간·획 데이터를 입력받는다. 종이 메모를 촬영한 사진에는 그 획 정보가 없으므로 사진 OCR의 대체 API로 사용할 수 없다. 한국어 지원 사실만으로 손글씨 메모 정확도까지 보장할 수는 없다. [11]

클라우드 OCR과 요구사항

Cloud Vision은 이미지 OCR을 제공하지만 이미지 데이터를 클라우드 API에서 처리하는 구성이다. 이 프로젝트의 원본 사진 미전송 조건을 유지하는 직접 연동 후보와는 구분해야 한다. 클라우드가 항상 더 정확하거나 기기 OCR이 항상 더 빠르다는 결론은 별도 실험 없이 내릴 수 없다. [12]

온디바이스 인식은 앱의 사진 저장·통신 금지를 자동 보장하지 않는다. 오류 보고, 디버그 로그, 임시 파일, 백업 경로를 포함해 사진 처리 흐름을 확인해야 한다.

4 기기 내 OCR 구현 방법

ML Kit 연동 예시

Flutter의 google_mlkit_text_recognition은 네이티브 ML Kit를 호출하는 커뮤니티 플러그인이며 Google이 직접 유지보수하는 Flutter SDK가 아니다. Android·iOS용으로 한국어 인식 의존성을 추가하고 Korean 스크립트를 선택한다. [8]

```
// google_mlkit_text_recognition 패키지를 import한 코드다.
Future<RecognizedText> readKoreanImage(String path) async {
  final recognizer = TextRecognizer(
    script: TextRecognitionScript.korean,
  );
  try {
    return await recognizer.processImage(
      InputImage.fromFilePath(path),
    );
  } finally {
    await recognizer.close();
  }
}
```

패키지 import를 생략한 설명용 Dart 코드 조각이다. 카메라 권한·중복 실행·취소·화면 종료·임시본 삭제는 호출 흐름에서 처리한다. 파일 경로는 앱이 관리하는 임시 사본을 사용하며 앨범 원본을 삭제하지 않는다.

모델 배포와 플랫폼 조건

Android ML Kit는 모델을 앱에 포함하는 방식과 Google Play Services를 통해 다운로드하는 방식을 구분한다. 포함 방식은 최초 실행 시 모델 준비가 가능하고, 다운로드 방식은 첫 사용 전 준비 여부를 확인해야 한다. 네트워크 없는 최초 실행은 별도 시험 대상이다. [7]

최소 OS는 프레임워크뿐 아니라 플러그인과 네이티브 SDK 조건의 교집합으로 정한다. Android 공식 설치 본문은 API 23 이상을 안내한다. 패키지 설명에 다른 최소값이 있어도 실제 고정한 SDK 조합의 설치 조건과 빌드 결과를 확인해야 한다. [7] [8]

상품 정보로 바꾸는 후처리

텍스트와 좌표에서 행·열 후보를 구성하고, 합계·세금·결제 정보 행을 제외한다. 메모는 상품명·수량, 영수증은 매장·날짜·상품별 수량·금액 초안으로 변환한다. 불확실한 값은 검토 대상으로 남긴다. F-15·F-18에 따라 사용자가 수정·제외한 결과만 확정한다.

Tesseract 구성안: 사진 입력 → 회전·대비 등 전처리 → kor 모델 인식 → 행·금액 후보 분석 → 검토 화면 순으로 구성할 수 있다. 분할 방식과 모델 종류의 효과는 같은 표본으로 비교해야 한다. [9] [10]

5 로컬 저장과 데이터 구성

설정 저장소는 단순 값 저장에 편리하지만 구매 기록과 변경 대기열의 무결성과는 구분해야 한다. shared_preferences 관리자는 쓰기 반환 후 디스크 영속화를 보장하지 않으며 중요한 데이터 저장에 사용하지 않도록 명시한다. 이전 조사에서 소개한 JSON 저장의 편의성이 N-05 통과를 뜻하지 않는다. [13]

SQLite는 트랜잭션으로 관련 변경을 함께 커밋하거나 되돌릴 수 있다. Flutter의 sqflite, Expo 환경의 expo-sqlite 등으로 접근할 수 있다. DB 자체의 원자성과 앱의 계정 분리·복구·마이그레이션은 별도로 검증한다. [14] [15] [16]

요구사항에서 도출한 공통 데이터 구성안

데이터	주요 정보	필요한 이유
장보기 항목	항목 ID, 사용자 ID, 상품명, 수량, 완료 상태, 버전, 삭제 표시	다른 항목 변경을 보존하고 수정·삭제 충돌 식별
가격 기록	기록 ID, 구매 ID, 상품·규격·할인 조건, 매장, 구매일, 수량, 상품별 결제 금액	부분 구매와 동일 조건 가격 비교
변경 대기열	요청 ID, 사용자 ID, 기준 버전, 변경 내용, 처리 상태	재실행 후 재전송과 중복 처리 방지
공개 가격	공개 ID와 허용된 가격 필드	다른 사용자에게 계정·개인 목록을 노출하지 않음

저장 순서 구성안

> 입력 검증 → 로컬 트랜잭션 → 화면 반영 → 서버 전송 → 서버 확인 후 대기열 정리

구매 확정에서는 완료 항목·잔여 수량·가격 기록·전송 요청을 하나의 로컬 트랜잭션에 묶는다. 서버가 실패해도 로컬 확정 기록과 전송 요청을 유지한다. 계정 전환 후 도착한 이전 계정 응답은 새 계정 저장소에 쓰지 않는다. 이 순서는 요구사항에 따른 구성안이며 특정 BaaS의 기본 보장은 아니다.

Flutter 공식 오프라인 설계도 로컬 저장과 원격 반영이 달라질 수 있음을 설명한다. 어떤 저장소를 사용하든 동기화 상태와 복구 방법을 앱에서 정의해야 한다. [17]

6 모바일 백엔드 서비스 비교

Firebase는 이 조사에서 Authentication과 Cloud Firestore를 중심으로 다룬다. Supabase는 PostgreSQL·Auth·Realtime·DB 함수를, AWS Amplify는 Cognito·AppSync와 Gen 1 DataStore 및 Gen 2의 구분을 중심으로 비교한다.

비교 항목	Firebase	Supabase	AWS Amplify
데이터	Firestore 문서·컬렉션	PostgreSQL 관계형 테이블	AppSync API, DynamoDB 등 연결
사용자 구분	Authentication + Security Rules	Auth + RLS	Cognito + API 권한
기본 오프라인	모바일 Firestore 캐시·대기 쓰기	로컬 DB·재전송 별도 구성	Gen 1 DataStore 지원; Gen 2는 별도 구성
충돌	동일 문서 마지막 쓰기 우선	버전 조건·DB 함수 등 설계	Gen 1 기본 Auto Merge; 사용자 선택은 추가 구현
구매 원자성	트랜잭션·일괄 쓰기	DB 트랜잭션·RPC	서버 사용자 정의 mutation·DB 트랜잭션
가격 조회	조건·정렬·인덱스 구성	SQL 필터·그룹·정렬	조회 키·인덱스·resolver 구성

서비스별 근거 [18] [19] [20] [23] [24] [25] [27] [31] [32] [34] [36]

요구사항 구현 가능 여부

세 서비스 모두 사용자 구분·데이터 저장·조회에 필요한 수단을 제공한다. F-11의 충돌 선택, F-12의 요청 중복 방지, F-22의 구매 일관성은 서비스의 이름만으로 충족되지 않는다. 해당 동작을 데이터 모델·서버 처리·로컬 대기열·화면에 연결하는 추가 개발이 필요하다.

가격 공유의 경계

개인 가격 기록에 사용자 식별자를 저장하더라도 비교 결과에는 허용된 가격 필드만 반환한다. 공유 여부만 확인하고 문서·행 전체를 반환하는 방식은 계정 식별 정보 제한을 위반할 수 있다. 공개용 데이터 또는 응답 모델을 두고 공유 해제 시 제외하도록 구성한다.

Amplify DataStore 조사 내용은 Gen 1 기능이다. 이를 Gen 2에 동일하게 적용할 수 있다는 의미로 읽어서는 안 된다. Gen 2 기능표는 DataStore 지원을 구분하고 있다. [34]

7 Firebase 목록 동기화 조사

기본 지원과 추가 구현

Firestore는 모바일에서 캐시 읽기와 오프라인 쓰기를 지원하고 재연결 후 변경을 반영한다. 같은 문서의 여러 변경은 기본적으로 마지막 쓰기가 우선하므로 사용자 선택형 충돌과는 다르다. 캐시에 있는 데이터와 모든 기록의 영구 보관도 구분한다. [19]

구현 구성안: 항목마다 버전을 보관하고 변경 요청의 기준 버전과 서버 버전을 비교한다. 충돌이면 로컬 변경과 서버 값을 모두 저장해 선택 화면을 연다. 삭제도 버전이 있는 삭제 표시로 전파한다. 재전송은 같은 요청 ID를 사용한다.

온라인 반영 시 버전 확인 예시

```
// 인증된 사용자의 항목 참조와 검증된 수량을 전달한다.
Future<void> updateQuantity(
  DocumentReference<Map<String, dynamic>> itemRef,
  int expectedVersion,
  int quantity,
) async {
  if (quantity < 1 || quantity > 999) {
    throw ArgumentError('수량 범위 오류');
  }
  await FirebaseFirestore.instance.runTransaction((tx) async {
    final snapshot = await tx.get(itemRef);
    final item = snapshot.data();
    if (item == null || item['version'] != expectedVersion ||
        item['deleted'] == true) {
      throw StateError('충돌 또는 삭제 확인 필요');
    }
    tx.update(itemRef, {
      'quantity': quantity,
      'version': expectedVersion + 1,
    });
  });
}
```

cloud_firestore 초기화와 import를 생략한 설명용 코드다. 트랜잭션은 오프라인에서 실패하므로 로컬 대기열을 저장한 뒤 온라인에서 호출하는 구성이다. 콜백은 재실행될 수 있으므로 그 안에서 화면·로컬 상태를 바꾸지 않는다. [20]

사용자 접근과 중복 요청

개인 목록은 users/{uid}/items/{itemId}처럼 구분하고 Security Rules에서 인증 UID와 경로를 대조한다. 소유자만 검사해서는 충분하지 않으므로 입력 범위와 버전 증가도 검증한다. 서버 SDK를 사용하는 API는 자체 인증·권한 검사가 필요하다. [18] [21]

예시는 요청 ID 저장을 생략했다. 응답 유실 후 재전송까지 처리하려면 처리 결과 문서를 같은 서버 트랜잭션에 저장하고 동일 요청의 기존 결과를 반환해야 한다.

8 Supabase 목록 동기화 조사

Supabase는 Flutter 클라이언트, PostgreSQL과 사용자 인증을 제공한다. Realtime의 DB 변경 구독은 온라인 변경을 받는 수단이다. 앱 재실행 후 오프라인 변경을 반영하는 저장·전송 대기열까지 같은 기능으로 간주하면 안 된다. [23] [25]

구현 구성안: SQLite에 항목과 대기 요청을 저장하고, 재연결 시 RPC로 기준 버전을 확인한다. Realtime은 새 변경 알림에 활용하며 연결 복구 시 누락 구간을 재조회한다. Brick 등 별도 오프라인 계층을 연동하는 방법도 공식 자료에 소개되어 있다. [26] [28]

RLS와 버전 조건 수정 예시

```
-- items 테이블과 기본 CRUD 권한이 준비된 설명용 예시다.
alter table public.items enable row level security;
create policy own_items on public.items
  for all to authenticated
  using ((select auth.uid()) = owner_id)
  with check ((select auth.uid()) = owner_id);

create function public.update_item_quantity(
  p_id uuid, p_version bigint, p_quantity integer
) returns setof public.items
language sql security invoker set search_path = '' as $$
  update public.items
  set quantity = p_quantity, version = version + 1
  where id = p_id and owner_id = auth.uid()
    and version = p_version and deleted = false
    and p_quantity between 1 and 999
  returning *;
$$;
revoke execute on function public.update_item_quantity(
  uuid, bigint, integer) from public, anon;
grant execute on function public.update_item_quantity(
  uuid, bigint, integer) to authenticated;
```

테이블 생성, CHECK 제약, 오류 응답 모델, 요청 중복 기록은 생략했다. 함수는 호출자 권한으로 실행한다. 반환 행이 없으면 유효성 오류·삭제·충돌·접근 제한을 확인하며, 0건을 동기화 성공으로 처리하지 않는다. [24] [27]

데이터 보존과 계정 변경

RLS는 서버 행 접근을 제한하고 로컬 DB의 계정 분리는 앱이 담당한다. 로그아웃 시 이전 계정의 전송을 멈추고 계정별 미전송 내용을 보존한다. 공유 조회는 개인 테이블 전체를 공개하지 않고 허용된 필드만 반환하는 API로 구성한다.

9 AWS Amplify 동기화 조사

Gen 1 DataStore와 Gen 2 구분

Gen 1 DataStore는 기기 저장소와 AppSync 동기화를 연결한다. 기본 Auto Merge는 단일 필드에서 서버 값을 우선하는 방식이며, Optimistic Concurrency나 Custom Lambda 등 다른 전략도 구성할 수 있다. [31] [32]

Gen 2의 Flutter 기능표는 DataStore와 해당 충돌 감지를 지원하지 않는 것으로 구분한다. 공식 이전 가이드는 Amplify API와 별도 로컬 캐시·오프라인 계층을 사용하는 방법을 설명한다. 따라서 AWS 후보는 'Gen 1 DataStore 구성'과 'Gen 2 API 구성'을 분리해 판단해야 한다. [34] [35]

Gen 1 목록 모델과 로컬 저장 예시

```
# Gen 1 GraphQL 모델 예시
type ShoppingItem @model @auth(rules: [{allow: owner}]) {
  id: ID!
  title: String!
  quantity: Int!
  completed: Boolean!
}
```

```
// 위 모델의 Dart 코드 생성과 Amplify 초기화 이후 호출한다.
await Amplify.DataStore.save(ShoppingItem(
  id: stableItemId,
  title: validatedTitle,
  quantity: validatedQuantity,
  completed: false,
));
```

데이터 조작 API를 보여 주는 설명용 코드다. stableItemId는 첫 생성 후 유지하는 항목 ID이며 검증값은 호출 전에 준비한다. @auth 구성은 해당 세대의 인증·모델 생성 설정과 함께 적용한다. save 완료를 모든 기기의 동기화 완료로 표시하지 않는다. [33] [34]

사용자 선택형 충돌 구성안

자동 병합 대신 충돌을 검출하는 전략을 구성하고, 앱의 별도 저장 영역에 두 후보와 원래 요청을 보존한다. conflictHandler 등에서 충돌을 수집해 화면에 전달하고 사용자가 선택한 뒤 다시 반영한다. 화면에서 선택할 때까지 장기 보존·삭제 충돌·중복 요청 상태를 유지하는 것은 추가 구현 범위다. [32]

Gen 2 API 구성에서는 SQLite·전송 대기열·버전 검증 mutation을 따로 구성할 수 있다. SDK와 백엔드 세대를 함께 기록해야 이전 예제를 현재 서비스의 기본 지원으로 오해하지 않는다.

10 구매 확정과 중복 방지 조사

핵심 기능 2: F-20~F-22는 목록 상태와 가격 기록이 함께 남는 구매 확정을 요구한다. 우유 3개 중 2개를 구매하면 2개 완료와 가격 기록을 남기고 1개 미완료를 보존한다. 아래 내용은 요구사항에 따른 서비스별 구현 구성안이다.

서비스	서버에서 함께 처리할 방법	추가로 보장할 조건
Firebase	트랜잭션으로 요청 ID 확인, 항목 버전 확인, 잔여·완료 항목과 가격 기록 저장	모든 읽기를 쓰기 전에 수행. 충돌이면 전체 거부. 동일 요청은 기존 결과 반환
Supabase	DB 함수 RPC의 한 트랜잭션에서 소유자·버전 확인과 목록·가격 저장	사용자+요청 ID 고유 제약. 동시 요청 직렬화·행 잠금 또는 조건 수정. 오류 시 롤백
AWS Amplify	사용자 정의 AppSync mutation과 DynamoDB 트랜잭션 등으로 다중 기록 저장	DataStore.save를 여러 번 호출한 것만으로 전체 구매 원자성을 가정하지 않음

원자적 서버 처리의 기술 근거 [20] [27] [30] [36]

공통 처리 흐름 예시

```
구매 확정(사용자, 요청 ID, 변경 묶음)  // 설명용 의사코드
  서버 트랜잭션 시작
  인증과 소유권 확인
  같은 요청의 저장 결과가 있으면 기존 결과 반환
  요청 ID를 예약하고 동일 요청의 동시 실행을 제어
  연결한 목록 항목들의 기준 버전과 현재 버전 확인
  충돌 또는 필수값 오류이면 전체 롤백
  구매분 완료와 잔여분, 가격 기록을 함께 저장
  동일 요청의 처리 결과 저장
  서버 트랜잭션 커밋
```

응답 유실과 영수증 중복의 차이

서버 저장 후 응답만 유실되면 같은 요청 ID로 재시도한다. 같은 요청 ID에 다른 내용을 보내면 오류로 처리하도록 내용 해시 등의 검증이 필요하다. 구매 건마다 안정적인 ID를 사용하되, ID 고정만으로 수정·완료 부작용의 중복까지 방지되는 것은 아니다.

다시 촬영한 영수증은 새 요청이므로 매장·구매일·상품·수량·금액으로 기존 구매 후보를 찾는다. 사용자가 별개 구매임을 확인했을 때만 새 기록을 만든다. 이 판정 정책은 D-04의 확정 대상이다.

서버 트랜잭션은 기기의 로컬 저장과 한 트랜잭션이 아니다. 로컬 확정 후에는 대기 상태를 유지하고, 서버 확인 뒤에만 동기화 완료로 바꾼다.

11 가격 조회와 공유 조사

핵심 기능 3: F-23~F-28은 본인 이력과 다른 사용자의 매장별 최근 구매 가격을 조회한다. 최근 기록은 구매일 내림차순, 같은 구매일이면 등록 시점 내림차순으로 정한다. 상품·규격·할인 조건이 다른 기록은 비교 순위에서 제외한다.

Firebase 조회 구성안

개인 기록과 공개 가격을 분리하고 공개 컬렉션에는 계정 정보를 넣지 않는다. 같은 비교 조건과 매장으로 필터링한 뒤 구매일·등록 시점을 정렬할 수 있다. 조합에 맞는 인덱스가 필요하다. [22]

```
// 한 매장에 대한 공개 가격 후보를 조회하는 Dart 예시다.
final latest = await FirebaseFirestore.instance
    .collection('public_prices')
    .where('comparisonKey', isEqualTo: comparisonKey)
    .where('storeId', isEqualTo: storeId)
    .orderBy('purchasedOn', descending: true)
    .orderBy('registeredAt', descending: true)
    .limit(1)
    .get();
```

공개 레코드만 이 컬렉션에 존재한다는 가정이다. 매장 목록과 인덱스 설정은 생략했다. 매장이 많아지면 매장별 최근 기록을 서버에서 관리하는 별도 집계 구성도 검토할 수 있다. 공유 해제 시 공개 사본 삭제와 최신 기록 재선정을 함께 처리해야 한다.

Supabase 조회 구성안

SQL의 DISTINCT ON으로 동일 조건에서 매장별 첫 기록을 고를 수 있다. 공개 응답은 SELECT 열을 명시하며 작성자 ID는 반환하지 않는다. [29]

```
-- 공개 가격 전용 테이블을 가정한 SQL 예시다.
select distinct on (store_id)
  public_id, store_id, product_name, purchased_on,
  package_size, unit, quantity, paid_amount, discount_key
from public.shared_prices
where comparison_key = :comparison_key
order by store_id, purchased_on desc, registered_at desc;
```

:comparison_key는 설명용 바인딩 변수다. 실제 DB 함수에서는 인자로 받는다. 테이블의 열 제약·공개 정책·읽기 권한 설정은 별도로 필요하다. 임의 권한의 DB 뷰를 만들었다고 개인 데이터 보호가 자동 적용된다고 가정하지 않는다.

12 AWS 가격 조회와 비교 계산

AWS Amplify 조회 구성안

공개 가격 모델을 개인 구매 모델과 분리한다. DynamoDB 기반 구성에서는 비교 조건과 매장을 조회 키로 만들고 구매일·등록 시점을 정렬 키에 포함하는 인덱스를 설계할 수 있다. AppSync에서 최신 1건 조회 API를 제공하고 매장별 결과를 묶어 반환한다. 이는 프로젝트용 구성안이며 자동으로 제공되는 '최저가 API'를 뜻하지 않는다.

처리	구현 방법	주의할 점
본인 이력	소유자 권한 모델을 구매일순으로 조회	다른 사용자의 개인 기록 접근 차단
매장별 최근 가격	비교 조건·매장 인덱스로 최신 기록 조회	등록 시점만으로 최근 구매일을 대신하지 않음
공개 API	허용된 필드만 담은 모델 또는 응답 반환	owner와 인증 식별자 등 내부 필드 제외
공유 해제	공개 기록 제거 후 최근 결과 재선정	클라이언트 캐시도 갱신하고 오래된 응답 반영 제어

Amplify 모델·인덱스·사용자 정의 query 지원 근거 [34]

가격 비교 계산은 앱 규칙

> 개당 가격 = 상품별 실제 결제 금액 ÷ 구매 개수
> 차액 = 비교 개당 가격 − 기준 개당 가격
> 변동률 = 차액 ÷ 기준 개당 가격 × 100

기준 구매가 2,500원, 비교 가격 2,300원이면 차액 −200원, 변동률 −8.0%다. 계산에는 반올림 전 값을 쓰고 표시할 때 가격·차액은 원 단위, 변동률은 소수 첫째 자리로 반올림한다. 기준 가격이 0원이면 변동률을 계산하지 않는다.

900mL와 1L, 서로 다른 묶음 수량·할인 조건은 같은 조건으로 자동 비교하지 않는다. 전체 영수증 쿠폰을 상품별로 나눌 수 없으면 참고용으로 남긴다. '기록 없음', '조회 실패', '비교 불가'를 다른 상태로 표시하고 과거 구매 가격을 현재 판매가로 안내하지 않는다.

동일 상품 확인과 AI의 범위

상품명 정규화나 유사도는 연결 후보 제시에 활용할 수 있으나 이름이 비슷한 상품을 자동 확정하지 않는다. 규격·단위·묶음·할인 조건을 사용자가 확인해야 한다. 상품 동등성 판단과 차액 계산은 OCR 모델의 기능과 구분하여 구현한다.

13 비용과 유지보수 조건

서비스 비용은 무료 여부만으로 비교하기 어렵다. 사용자 수, 기기 수, 동기화 빈도, 조회되는 기록 수와 서버 함수 호출량을 같은 가정으로 놓아야 한다. 아래는 가격표의 과금 요소를 정리한 것으로 특정 요금이나 월 비용을 확정한 견적은 아니다.

서비스	주요 과금 요소	이 앱에서 늘어날 수 있는 사용량
Firebase	Firestore 읽기·쓰기·삭제, 저장·전송, 함수와 인증 방식별 비용	실시간 구독과 재조회, 공개 가격 갱신, 구매 트랜잭션 재시도
Supabase	요금제·DB 컴퓨트, 저장·전송, 활성 사용자·Realtime 등 할당량	가격 조회와 DB 함수, 기기별 변경 구독, 별도 동기화 서비스
AWS Amplify	Cognito·AppSync·DynamoDB·Lambda 등 구성한 AWS 서비스별 과금	구독·API 요청, 트랜잭션, 인덱스와 함수 실행

공식 요금 정보 [37] [38] [39] [40]

공통 견적 입력 예시

예를 들어 월 사용자 100명·사용자당 기기 2대·하루 목록 변경 10건·하루 가격 조회 5회처럼 공통 가정을 정할 수 있다. 이 수치는 조사용 예시이며 실제 예상 수요가 아니다. 한 화면 조회에 읽는 레코드 수, 동기화 때 전송하는 변경 수와 재시도율을 추가해 사용량으로 환산한다.

원본 사진을 업로드하지 않으므로 사진 저장 비용을 필수 기능 견적에 넣지 않는다. 공개 가격의 중복 저장, 재접속 구독, 인덱스 읽기, 별도 오프라인 연동 서비스 비용은 선택한 구성에 따라 달라진다. 지역·요금제·무료 할당 조건은 채택 시점의 공식 가격표로 다시 계산한다.

플랫폼 종속성과 유지보수

Firestore의 문서·규칙·쿼리, Supabase의 SQL·RLS·RPC, Amplify의 모델 생성·AppSync 권한·AWS 인프라 설정은 각각 다른 이전 비용을 만든다. PostgreSQL 사용만으로 인증과 실시간 API까지 그대로 이전되는 것은 아니다.

OCR은 네이티브 SDK와 Flutter 플러그인의 버전 호환성을 함께 관리한다. Tesseract는 엔진·모델·모바일 래퍼의 호환성을 확인한다. 플랫폼 최소 버전, SDK 업데이트, 생성 코드, 데이터 이전 시험은 최종 채택의 유지보수 조건이다.

14 후속 검증과 조사 결과

채택 판단 전에 확보할 증빙

검증 항목	시험 방법	요구사항의 기준
저장	오프라인 변경 100건 후 강제 종료·재실행 10회	저장 완료 데이터·대기 변경 누락 0건
동기화	두 기기, 단절·재연결, 동일 요청 3회 전송	중복·변경 누락·삭제 항목 부활 없음
충돌	같은 항목 수정, 수정·삭제, 전송 중 추가 수정	양쪽 보존, 사용자 선택 후 최종 상태 일치
구매 확정	부분 구매, 로컬 실패, 서버 실패·응답 유실	완료 상태·가격 일치, 가격 중복 없음
OCR	인쇄·손글씨·흐림·빈 사진·권한 거부·취소	검토 후 선택한 항목만 저장, 사진 미전송
가격 비교	구매일·등록일 역전, 조건 불일치, 공유 해제	최근 구매일 선택, 조건별 분리, 공개 정보 제한

성능 제안값은 앱 시작 3초, 목록 동작 1초, 변경 100건 동기화 10초, 준비된 기기의 OCR 10초, 정상 가격 조회 5초다. 요구사항에 정한 반복 횟수·통과 비율과 기준 기기를 함께 확정한다. OCR 정확도 합격선은 D-06에서 정한다.

조사 결과

Flutter와 React Native는 목록 화면과 사진 입력을 구성할 수 있으며, 한국어 OCR의 플랫폼 연동은 별도 조건이다. ML Kit와 Tesseract는 기기 내 이미지 인식 후보지만 손글씨 메모와 영수증 필드 추출의 정확도는 프로젝트 표본으로 확인해야 한다.

Firebase·Supabase·AWS Amplify는 백엔드 구현에 활용할 수 있는 저장·인증·조회 수단을 제공한다. 오프라인 기본 지원과 데이터 모델은 다르며, 세 서비스 모두 요구사항에 맞는 충돌 선택·구매 확정·공개 가격 관리를 추가 설계해야 한다. 특히 Amplify는 세대별 기능 차이를 유지해야 한다.

최종 채택은 기능 시연, 실패 흐름 시험, 동일 조건 비용 산정과 팀의 정책 확정 결과를 근거로 결정한다. 본 조사에서는 기술 채택, 앱 시험 통과, OCR 정확도나 서비스 성능 우열을 확정하지 않는다.

15 프로젝트 자료와 공식 출처

프로젝트와 강의 자료

1. 모바일 앱 프로젝트 제안서 — 목표·주요 기능·기술적 어려움·구현 범위.
2. 얼마였지 요구사항 분석서 v1.0 — HM-REQ-001, 2026년 9월 14일. F-01~F-28, N-01~N-10, C-01~C-09, D-01~D-06, T-01~T-12.
3. 얼마였지 UI 프로토타입 설계서 v1.0 — UI-SPEC v1.0, 2026년 9월 21일. 핵심 화면 8종과 오류·복구 흐름.
4. 모바일 백엔드 서비스 — 허준영, 한성대학교. 8쪽 팀 활동의 서비스 2개 이상 비교와 핵심 기능 2개 이상 조사.
5. 얼마였지 크로스플랫폼 조사 초안 v36.0 — 기존 목록·사진 기능 비교 자료.

공식 출처 읽는 방법

본문 [번호]는 아래 공식 문서로 연결된다. 모두 2026년 10월 9일 확인했다. 온라인 문서와 가격표는 갱신될 수 있으므로 SDK·플러그인 버전과 채택 시점의 정책을 다시 확인한다. 서비스 소개 자료만으로 요구사항 시험 통과를 판단하지 않는다.

모바일 SW와 AI

[1] Flutter architectural overview
https://docs.flutter.dev/resources/architectural-overview

[2] React Native Core Components and Native Components
https://reactnative.dev/docs/intro-react-native-components

[3] Flutter image_picker
https://pub.dev/packages/image_picker

[4] Expo ImagePicker
https://docs.expo.dev/versions/latest/sdk/imagepicker/

[5] Google ML Kit
https://developers.google.com/ml-kit

[6] ML Kit Text Recognition v2 지원 언어
https://developers.google.com/ml-kit/vision/text-recognition/v2/languages

[7] ML Kit Android 이미지 텍스트 인식
https://developers.google.com/ml-kit/vision/text-recognition/v2/android

[8] Flutter google_mlkit_text_recognition
https://pub.dev/packages/google_mlkit_text_recognition

[9] Tesseract User Manual
https://tesseract-ocr.github.io/tessdoc/

[10] Tesseract 언어별 모델
https://tesseract-ocr.github.io/tessdoc/Data-Files-in-different-versions.html

[11] ML Kit Digital Ink Recognition
https://developers.google.com/ml-kit/vision/digital-ink-recognition

[12] Google Cloud Vision 이미지 OCR
https://docs.cloud.google.com/vision/docs/ocr

16 공식 출처 로컬 저장과 백엔드

문서 확인일 2026년 10월 9일 · 번호는 본문 인용과 연결된다.

[13] Flutter shared_preferences
https://pub.dev/packages/shared_preferences

[14] SQLite Is Transactional
https://www.sqlite.org/transactional.html

[15] Flutter sqflite
https://pub.dev/packages/sqflite

[16] Expo SQLite
https://docs.expo.dev/versions/latest/sdk/sqlite/

[17] Flutter Offline first support
https://docs.flutter.dev/app-architecture/design-patterns/offline-first

[18] Firebase Authentication Flutter 시작
https://firebase.google.com/docs/auth/flutter/start

[19] Firestore 오프라인 데이터 접근
https://firebase.google.com/docs/firestore/manage-data/enable-offline

[20] Firestore 트랜잭션과 일괄 쓰기
https://firebase.google.com/docs/firestore/manage-data/transactions

[21] Firestore Security Rules 조건
https://firebase.google.com/docs/firestore/security/rules-conditions

[22] Firestore 단순 및 복합 쿼리
https://firebase.google.com/docs/firestore/query-data/queries

[23] Supabase Flutter 시작
https://supabase.com/docs/guides/getting-started/quickstarts/flutter

[24] Supabase Row Level Security
https://supabase.com/docs/guides/database/postgres/row-level-security

[25] Supabase Postgres Changes
https://supabase.com/docs/guides/realtime/postgres-changes

[26] Supabase와 Flutter 및 Brick 오프라인 구현
https://supabase.com/blog/offline-first-flutter-apps

[27] Supabase Database Functions
https://supabase.com/docs/guides/database/functions

17 공식 출처 API와 비용

문서 확인일 2026년 10월 9일 · 번호는 본문 인용과 연결된다.

[28] Supabase Flutter RPC
https://supabase.com/docs/reference/dart/rpc

[29] PostgreSQL SELECT
https://www.postgresql.org/docs/current/sql-select.html

[30] PostgreSQL Transactions
https://www.postgresql.org/docs/current/tutorial-transactions.html

[31] Amplify Flutter Gen 1 DataStore
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/

[32] Amplify Flutter DataStore 충돌 처리
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/conflict-resolution/

[33] Amplify Flutter DataStore 데이터 조작
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/manipulate-data/

[34] Amplify Flutter Gen 2 Feature Parity
https://docs.amplify.aws/flutter/start/migrate-to-gen2/feature-matrix/

[35] Amplify Flutter DataStore에서 이전
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/migrate-from-datastore/

[36] AppSync DynamoDB 트랜잭션
https://docs.aws.amazon.com/appsync/latest/devguide/tutorial-dynamodb-transact.html

[37] Firebase 요금제
https://firebase.google.com/pricing

[38] Firestore 과금 구조
https://firebase.google.com/docs/firestore/pricing

[39] Supabase 요금제
https://supabase.com/pricing

[40] AWS Amplify 요금 구조
https://aws.amazon.com/amplify/pricing/